TERRAFORM CHALLENGE

Terraform Project: AWS EC2 + Docker Compose + Nginx Reverse Proxy + Log Rotation

Complete practical project documentation

1. Project introduction – What are we building?

In this project, we will deploy an application on an AWS EC2 instance using Terraform.

We will use Terraform, AWS EC2, Ubuntu, Docker, Docker Compose, Nginx and Logrotate to deploy and manage the application.

The main objective is to access the Docker application through port `8080`, even though the Docker application itself runs on port `8000`.

1.1 Why are we doing this project?

In real-time DevOps projects, applications are commonly deployed inside Docker containers. However, users should not always need to access the container's port directly.

We use a web server such as Nginx as a reverse proxy.

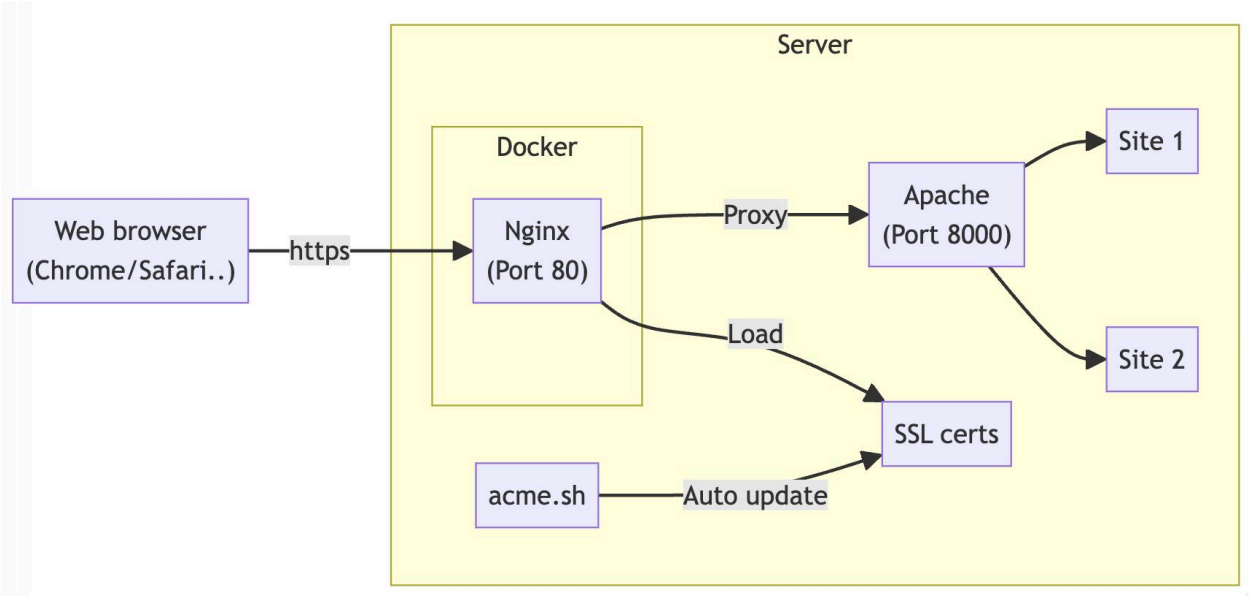
For example:

- Docker application runs on port `8000`.
- Nginx listens on port `8080`.
- When a user accesses port `8080`, Nginx forwards the request to port `8000`.
- Nginx records access logs in `/var/log/cb.log`.
- Logrotate manages the log file to prevent it from consuming too much disk space.

1.2 What is the use of each technology?

Technology	Why are we using it?
Terraform	Automatically creates AWS infrastructure.
AWS EC2	Provides a virtual server to run our application.
Ubuntu	Operating system for the EC2 instance.
Docker	Runs the application inside a container.
Docker Compose	Defines and starts the application using a YAML file.
Nginx	Acts as a reverse proxy and listens on port 8080.
Log files	Store application access information.
Logrotate	Automatically rotates and manages log files.

1.3 Architecture – How does the project work?



The request flow is:

1. User opens `http://EC2-PUBLIC-IP:8080`.
2. AWS Security Group allows port 8080.
3. Nginx receives the request on port 8080.
4. Nginx forwards the request to `http://127.0.0.1:8000`.
5. Docker container processes the request.
6. Docker application sends the response to Nginx.
7. Nginx returns the response to the user.
8. Nginx saves access logs in `/var/log/cb.log`.
9. Logrotate periodically rotates the log file.

Important: Port 8000 will only be accessible locally on the EC2 instance. Users will access the application through port 8080.

2. Project requirements and port explanation

Before writing Terraform code, understand the port configuration.

Port configuration

8080

Nginx

Public access port

8000

Docker host port

Local access port

8080

Container port

Application port

Request flow: Internet → EC2:8080 → Nginx → 127.0.0.1:8000 → Docker container:8080

Why are there two different port numbers?

The repository's README explains that its Hello World application listens on container port `8080`. We publish that container port on host port `8000` using Docker. Nginx then forwards requests from port `8080` to host port `8000`.

GitHub+1

We will use the repository's published Docker image:

`ghcr.io/infrastructure-as-code/hello-world`

The repository also provides `/`, `/health` and `/metrics` endpoints.

GitHub

3. Prerequisites

Before starting, you need:

1. An AWS account.
2. Terraform installed on your Windows machine.
3. AWS CLI installed and configured.
4. VS Code installed.
5. An AWS region in which the supplied Ubuntu AMI is available.
6. An EC2 key pair if you want SSH access.

Important AMI note: AMI IDs are region-specific. I will use your supplied AMI ID, `ami-0461b1436c9b3c1a3`, and set the example region to `us-east-1`. Verify that this AMI exists in that region before applying the configuration.

Check your installations

Open PowerShell.

```
None
terraform -version
aws --version
```

```
Terraform v1.16.6
on windows_amd64
+ provider registry.terraform.io/hashicorp/aws v5.100.0

Your version of Terraform is out of date! The latest version
is 1.17.1. You can update by downloading from https://developer.hashicorp.com/terraform/install
```

```
aws-cli/2.36.30 Python/3.14.6 Windows/11 exe/AMD64
```

Configure AWS credentials if you have not already done so:

```
None
aws configure
```

Enter your AWS Access Key, Secret Key, region and output format.

Verify your AWS identity:

```
None
aws sts get-caller-identity
```

```
PS C:\terraform-workspaces\terraform-vpc-project> aws sts get-caller-identity
{
    "UserId": "686194638896",
    "Account": "686194638896",
    "Arn": "arn:aws:iam::686194638896:root"
}
```

4. Create the project directory in VS Code

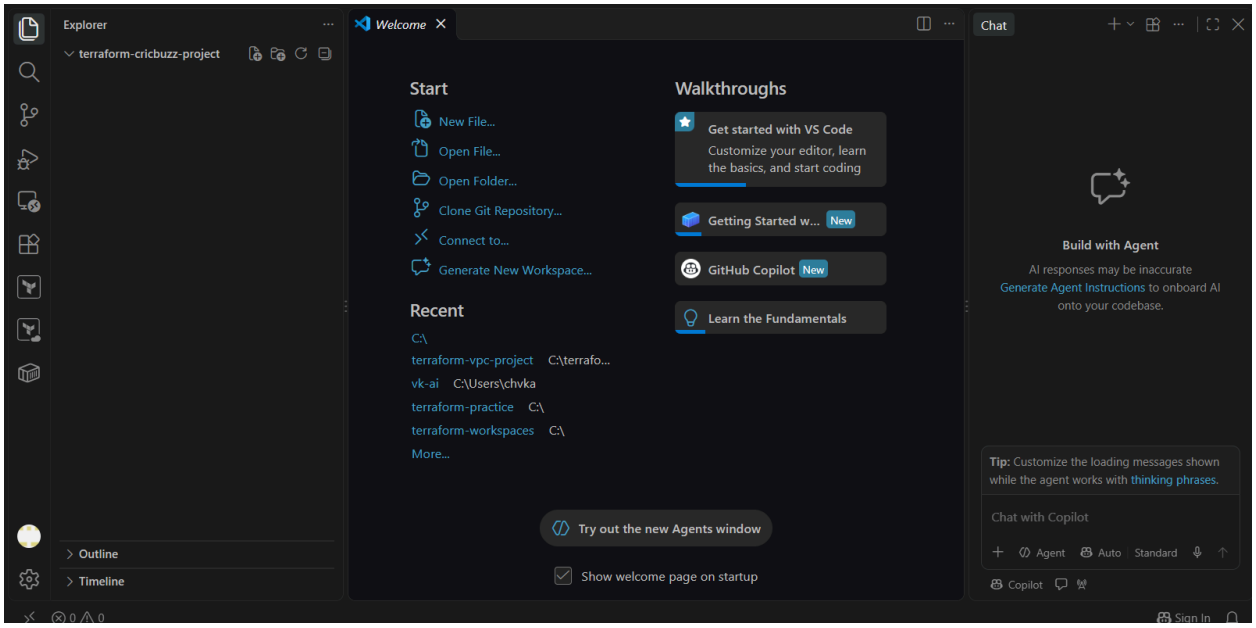
We will create all required Terraform and application configuration files in one folder.

Step 1: Create a folder

Open PowerShell and execute:

```
None
mkdir terraform-cricbuzz-project
cd .\terraform-cricbuzz-project
code .
```

VS Code will open the project folder.



Step 2: Create these files

In the VS Code Explorer (left side):

Right-click the project folder → New File.

Create the following files:

terraform-cricbuzz-project

main.tf

AWS provider, EC2 and security group

variables.tf

Project input variables

outputs.tf

EC2 public IP and application URL

user_data.sh

Automatically installs and configures software

docker-compose.yml

Runs the Docker application

nginx.conf

Configures reverse proxy and access logs

cb-logrotate

Configures log rotation

We will now write the code in each file.

5. Terraform configuration

Step 1: main.tf

Purpose: This file creates AWS resources.

It creates:

- AWS provider configuration.
- Security Group.
- EC2 instance.
- Ubuntu server.
- Automatic installation through the user-data script.

Open `main.tf` and paste this code:

```
None
```

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}

# Create Security Group
resource "aws_security_group" "cricbuzz_sg" {
  name        = "cricbuzz-web-sg"
  description = "Allow HTTP application and restricted SSH"

  # Public application access through Nginx
  ingress {
    description = "Nginx application port"
    from_port   = 8080
    to_port     = 8080
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  # SSH only from the administrator's IP
  dynamic "ingress" {
    for_each = var.admin_cidr == "" ? [] : [var.admin_cidr]

    content {
      description = "Restricted SSH access"
      from_port   = 22
      to_port     = 22
      protocol    = "tcp"
```

```
      cidr_blocks = [ingress.value]
    }
  }

  egress {
    description = "Allow outbound traffic"
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }

  tags = {
    Name = "cricbuzz-security-group"
  }
}

# Create EC2 Instance
resource "aws_instance" "cricbuzz_server" {
  ami                    = var.ami_id
  instance_type          = var.instance_type
  vpc_security_group_ids = [aws_security_group.cricbuzz_sg.id]
  key_name               = var.key_name != "" ? var.key_name : null


user_data = templatefile("${path.module}/user_data.sh", {
  project_dir      = "/opt/cricbuzz"
  compose_yaml     = file("${path.module}/docker-compose.yml")
  nginx_config     = file("${path.module}/nginx.conf")
  logrotate_config = file("${path.module}/cb-logrotate")
})

  user_data_replace_on_change = true

  tags = {
    Name        = "Cricbuzz-Docker-Server"
```

```
    Environment = "Dev"
    ManagedBy   = "Terraform"
  }
}
```

Explanation:

Resource	Purpose
`terraform`	Specifies the required AWS provider.
`provider "aws"`	Connects Terraform to AWS.
`aws_security_group`	Controls inbound and outbound traffic.
`ingress`	Allows incoming connections.
`egress`	Allows outgoing connections.
`aws_instance`	Creates the EC2 instance.
`ami`	Specifies the Ubuntu machine image.
`instance_type`	Specifies the EC2 size.

`user_data`	Executes the installation script during instance initialization.
`tags`	Adds identifying information to AWS resources.

Why do we allow port 8080?

Because Nginx listens on port 8080 and external users need to access the application.

Why don't we allow port 8000 publicly?

Because Docker's application port should only be accessible locally. Nginx handles public requests.

Step 2: variables.tf

Purpose: Stores the values that we may need to change later without editing the main Terraform code.

Paste this:

```
None

variable "aws_region" {
  description = "AWS deployment region"
  type        = string
  default     = "us-east-1"
}

variable "ami_id" {
  description = "Ubuntu AMI ID"
  type        = string
  default     = "ami-0461b1436c9b3c1a3"
}

variable "instance_type" {
  description = "EC2 instance type"
  type        = string
```

```
  default     = "t2.micro"
}

variable "key_name" {
  description = "Existing EC2 key pair name; empty disables SSH key
assignment"
  type        = string
  default     = ""
}

variable "admin_cidr" {
  description = "Administrator public IP in CIDR format for SSH"
  type        = string
  default     = ""
}
```

Explanation:

- `aws_region`: Region where EC2 will be created.
- `ami_id`: Ubuntu AMI ID.
- `instance_type`: EC2 instance size.
- `key_name`: Existing AWS key pair name, if SSH access is required.
- `admin_cidr`: Your public IP address in CIDR format, for example `203.0.113.10/32`.

If you want SSH access, change the last two defaults appropriately. Do not open SSH to everyone in a production environment.

Step 3: outputs.tf

Purpose: Displays the EC2 public IP and application URL after Terraform creates the instance.

Paste this:

```
None

output "ec2_instance_id" {
  description = "Created EC2 instance ID"
  value       = aws_instance.cricbuzz_server.id
```

```
}

output "ec2_public_ip" {
  description = "Public IP address of EC2"
  value       = aws_instance.cricbuzz_server.public_ip
}

output "application_url" {
  description = "Cricbuzz application URL"
  value       =
"http://${aws_instance.cricbuzz_server.public_ip}:8080"
}
```

6. Create the Docker Compose file

Step 4: docker-compose.yml

Purpose: Docker Compose defines how our application container should run.

We are using the Docker image published by the given GitHub repository.

Paste this into `docker-compose.yml`:

```
None

services:
  cricbuzz:
    image: ghcr.io/infrastructure-as-code/hello-world:latest

    container_name: cricbuzz-app

    ports:
      - "127.0.0.1:8000:8080"

    restart: unless-stopped
```

Explanation of each line:

Configuration	Purpose
`services`	Defines application services.
`cricbuzz`	Service name.
`image`	Docker image to download and run.
`container_name`	Gives the container a fixed name.
`127.0.0.1:8000:8080`	Maps EC2 localhost port 8000 to container port 8080.
`restart`	Automatically restarts the container when needed.

Important port explanation:

```
None
- "127.0.0.1:8000:8080"
```

- `127.0.0.1` – Only the EC2 server itself can access this published port.
- First `8000` – Host port.
- Second `8080` – Docker container port.

We are not exposing port 8000 to the public internet.

7. Configure Nginx as a reverse proxy

Step 5: nginx.conf

Purpose: Nginx receives requests on port 8080 and forwards them to the Docker application on port 8000.

It also saves access logs in `/var/log/cb.log`.

Paste this into `nginx.conf`:

```
None

log_format cb_format
    '$remote_addr - $remote_user [$time_local] '
    '"$request" $status $body_bytes_sent '
    '"$http_referer" "$http_user_agent"';

server {
    listen 8080;
    server_name _;

    access_log /var/log/cb.log cb_format;
    error_log /var/log/cb-error.log warn;

    location / {
        proxy_pass http://127.0.0.1:8000;

        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For
$proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Explanation:

Configuration	Purpose

`listen 8080`	Nginx listens on port 8080.
`server_name _`	Accepts requests for the default server.
`log_format`	Defines the access log format.
`access_log`	Saves access logs in `/var/log/cb.log`.
`error_log`	Saves Nginx errors separately.
`location /`	Handles incoming application requests.
`proxy_pass`	Forwards requests to Docker on port 8000.
`proxy_set_header`	Preserves client and request information.

Why use a reverse proxy?

Imagine that a customer visits your website.

The customer does not need to know which Docker container is running behind the application. Nginx receives the request and forwards it to the correct service.

This is the purpose of a reverse proxy.

NGINX Documentation+1

8. Configure Logrotate

Step 6: cb-logrotate

Purpose: Log files continuously grow when users access an application.

For example, if 10,000 users access the application, Nginx generates many access log entries.

Without log rotation, log files can consume unnecessary disk space.

Logrotate automatically manages the log file by:

- Rotating old logs.
- Compressing previous logs.
- Retaining a limited number of backups.
- Removing older backups.

Create `cb-logrotate` and paste:

```
None

/var/log/cb.log {
    daily
    rotate 7
    compress
    delaycompress
    missingok
    notifempty
    copytruncate
    create 0644 root adm
}
```

Explanation:

Directive	Purpose
`/var/log/cb.log`	Log file to manage.

`daily`	Checks for rotation daily.
`rotate 7`	Keeps seven rotated copies.
`compress`	Compresses rotated logs.
`delaycompress`	Compresses the previous rotation on the next cycle.
`missingok`	Does not report an error if the log is missing.
`notifempty`	Does not rotate an empty file.
`copytruncate`	Copies the current log and truncates the original file.
`create`	Specifies permissions and ownership for the recreated file.

For example, rotated files may look like this:

```
None
/var/log/cb.log
/var/log/cb.log.1
/var/log/cb.log.2.gz
/var/log/cb.log.3.gz
```

Logrotate normally runs through the system's scheduled daily job.

Ubuntu Manpages+1

9. Automatically install everything using Terraform

Step 7: user_data.sh

This is one of the most important files in the project.

Why?

Normally, after creating an EC2 instance, we manually connect to Ubuntu and install:

1. Docker.
2. Docker Compose.
3. Nginx.
4. Logrotate.
5. Application container.

But we want Terraform to perform these tasks automatically.

We use EC2 User Data to execute a Bash script when the instance starts for the first time.

The script will:

- Update Ubuntu packages.
- Install Nginx and Logrotate.
- Install Docker and Docker Compose plugin.
- Create the application directory.
- Create the Docker Compose configuration.
- Start the Docker container.
- Configure Nginx.
- Create the log file.
- Configure Logrotate.
- Start Nginx.

Paste this code into `user_data.sh`:

```
None

#!/bin/bash

set -euxo pipefail

# Update Ubuntu packages
apt-get update -y
```

```
# Install required packages
apt-get install -y ca-certificates curl nginx logrotate

# Install Docker and Docker Compose plugin
curl -fsSL https://get.docker.com -o /tmp/get-docker.sh
sh /tmp/get-docker.sh

# Enable Docker
systemctl enable docker
systemctl start docker

# Create application directory
mkdir -p ${project_dir}

# Create Docker Compose file
cat > ${project_dir}/docker-compose.yml <<'COMPOSE_EOF'
${compose_yaml}
COMPOSE_EOF

# Start Docker application
cd ${project_dir}

docker compose pull
docker compose up -d

# Configure Nginx
cat > /etc/nginx/conf.d/cricbuzz.conf <<'NGINX_EOF'
${nginx_config}
NGINX_EOF

# Remove default Nginx site if enabled
rm -f /etc/nginx/sites-enabled/default

# Create application access log
touch /var/log/cb.log
```

```
chown root:adm /var/log/cb.log
chmod 0644 /var/log/cb.log

# Configure Logrotate
cat > /etc/logrotate.d/cb-logrotate <<'LOGROTATE_EOF'
${logrotate_config}
LOGROTATE_EOF

# Test Nginx configuration
nginx -t

# Enable and restart Nginx
systemctl enable nginx
systemctl restart nginx

# Save installation completion status
echo "Cricbuzz deployment completed successfully" \
    > /var/log/cricbuzz-deployment.log
```

Important: The `${...}` expressions in this file are Terraform template variables. Terraform will replace them with the contents of your three configuration files before EC2 executes the script.

We must now connect those files to the user-data template.

Step 8: Update main.tf user_data

In your existing `aws_instance` resource, replace the current `user_data` block with this:

None

```
user_data = templatefile("${path.module}/user_data.sh", {
  project_dir     = "/opt/cricbuzz"
  compose_yaml    = file("${path.module}/docker-compose.yml")
  nginx_config    = file("${path.module}/nginx.conf")
  logrotate_config = file("${path.module}/cb-logrotate")
})
```

Keep the rest of the EC2 resource unchanged.

Now Terraform will read the three configuration files and pass their contents into `user_data.sh`.

This means you do not need to manually SSH into EC2 to create these configurations.

10. Final project file structure

Before executing Terraform, verify that your VS Code Explorer looks like this:

C:\terraform-cricbuzz-project

main.tf

variables.tf

outputs.tf

user_data.sh

docker-compose.yml

nginx.conf

cb-logrotate

You should have seven files.

11. Execute Terraform step by step

Now we will deploy the complete project into AWS.

Step 1: Open the VS Code terminal

In VS Code:

1. Click Terminal in the top menu.
2. Click New Terminal.
3. Verify that you are inside the project directory.

```
None
cd C:\terraform-cricbuzz-project
```

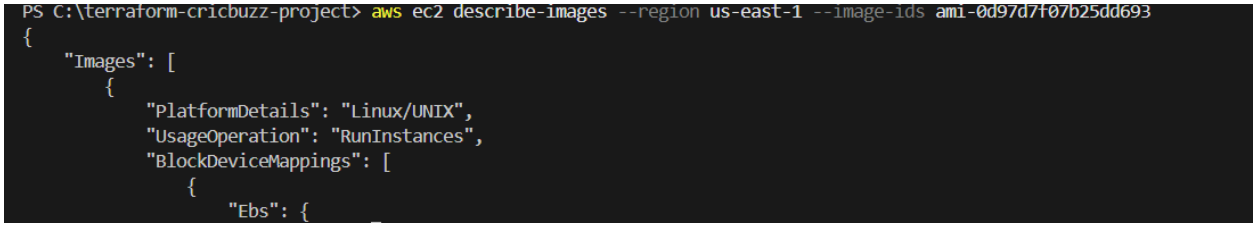
Step 2: Verify the AMI

Run:

```
None
aws ec2 describe-images --region us-east-1 --image-ids
ami-0461b1436c9b3c1a3
```

If the image is returned, verify its architecture and Ubuntu version.

If you receive `InvalidAMIID.NotFound`, find an Ubuntu AMI available in your selected region and replace the AMI variable.



Step 3: Initialize Terraform

```
None
terraform init
```

Why?

Downloads the AWS provider and prepares Terraform to work with AWS.

Expected result:

```
None
Terraform has been successfully initialized!
```

```
PS C:\terraform-cricbuzz-project> terraform init
Initializing the backend...

Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 6.0"...
- Installing hashicorp/aws v6.71.0...
- Installed hashicorp/aws v6.71.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

Step 4: Validate Terraform

```
None
terraform validate
```

Why?

Checks whether your Terraform configuration has syntax and configuration errors.

Expected result:

```
None
Success! The configuration is valid.
```

```
PS C:\terraform-cricbuzz-project> terraform validate
Success! The configuration is valid.

PS C:\terraform-cricbuzz-project>
```

Step 5: Format Terraform code

```
None
terraform fmt
```

Why?

Formats Terraform files into a standard readable format.

```
PS C:\terraform-cricbuzz-project> terraform fmt
main.tf
```

Step 6: Generate Terraform execution plan

```
None
terraform plan
```

Why?

Shows what Terraform will create before making changes to AWS.

Expected output should indicate approximately:

```
None
Plan: 2 to add, 0 to change, 0 to destroy.
```

```
      + vpc_id                 = (known after apply)
    }

Plan: 2 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + application_url = (known after apply)
  + ec2_instance_id = (known after apply)
  + ec2_public_ip   = (known after apply)
```

The two resources are:

1. Security Group.
2. EC2 Instance.

Step 7: Create EC2 instance

```
None
terraform apply
```

Terraform displays the execution plan again.

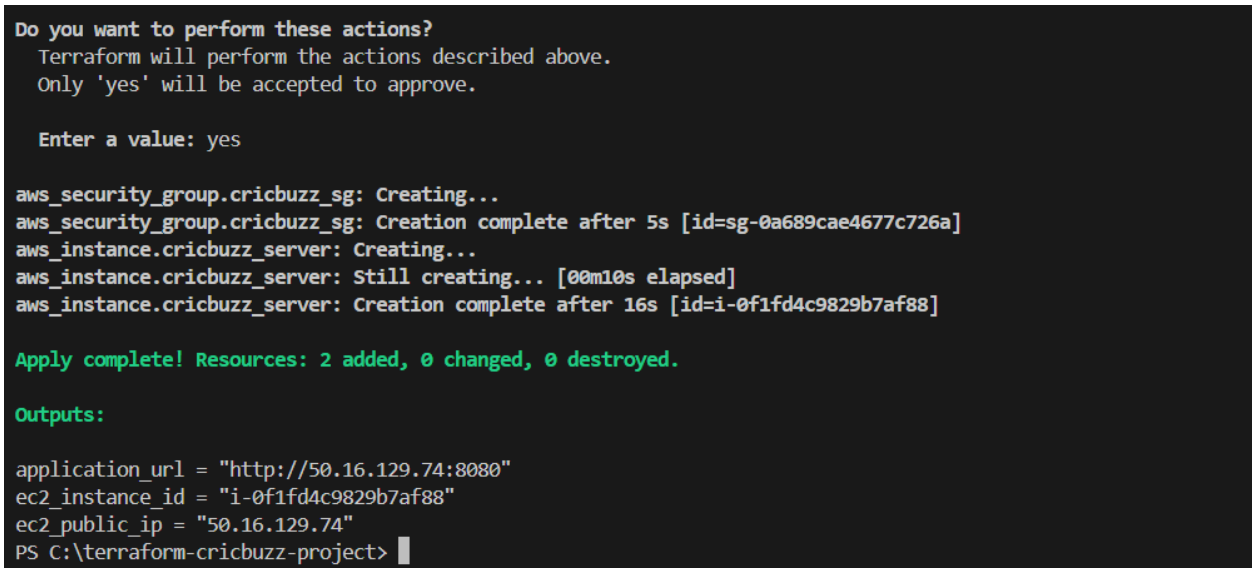
Enter:

```
None
yes
```

Terraform will create the resources.

Deployment initiated

Wait for Terraform to finish. EC2 creation and the installation script are separate processes, so Terraform finishing does not necessarily mean that Nginx and Docker are already ready.



Step 8: Get the public IP

```
None
terraform output
```

Expected output structure:

```
None
application_url = "http://PUBLIC-IP:8080"
ec2_instance_id = "i-xxxxxxxxxxxxxxxxx"
ec2_public_ip = "PUBLIC-IP"
```

Copy the public IP.

```
PS C:\terraform-cricbuzz-project> terraform output
application_url = "http://50.16.129.74:8080"
ec2_instance_id = "i-0f1fd4c9829b7af88"
ec2_public_ip = "50.16.129.74"
```

12. Verify the project in AWS Console

Now we will verify that Terraform created the EC2 instance.

Step 1: Open AWS Console

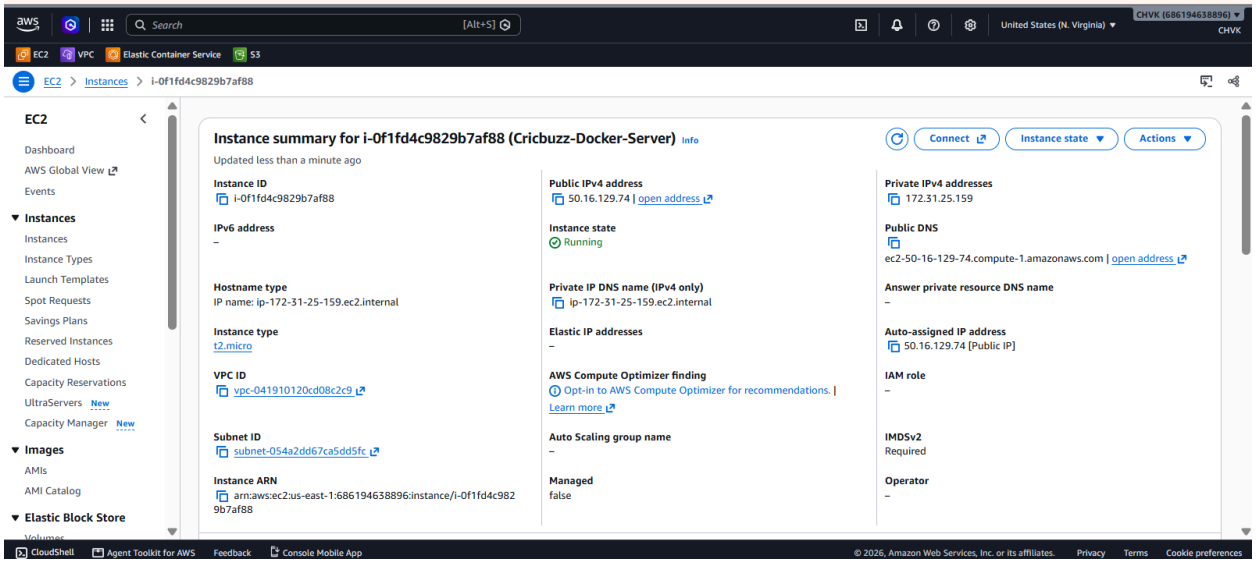
Visit AWS EC2 Console.

Step 2: Navigate to Instances

1. Select your region (`us-east-1`).
2. Click Instances.
3. Find `Cricbuzz-Docker-Server`.
4. Check the instance state.

Expected:

Running

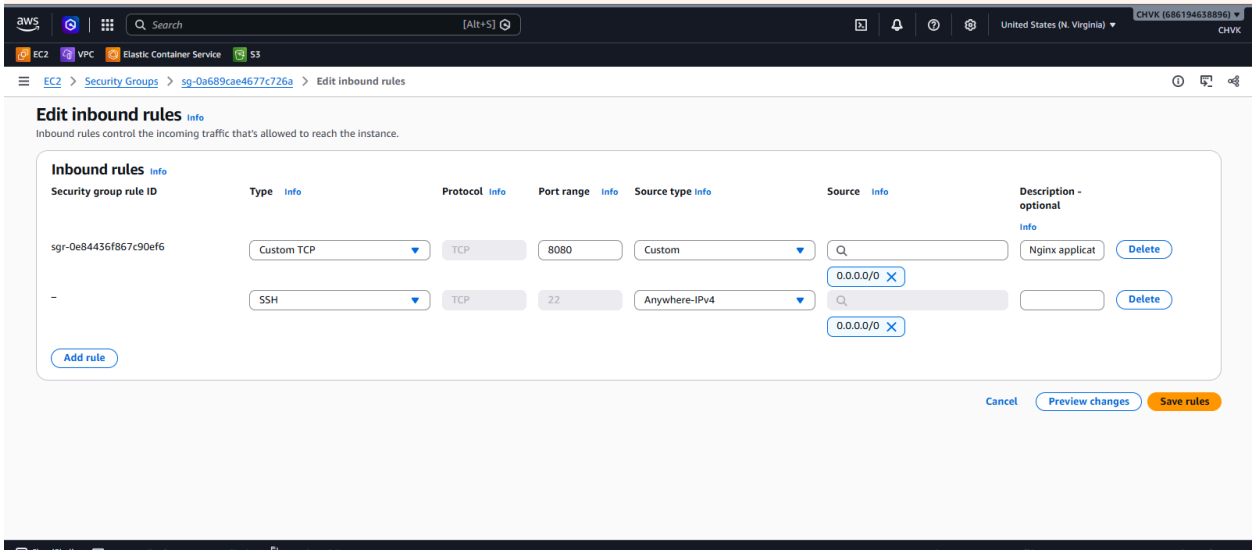


Step 3: Verify Security Group

1. Select your EC2 instance.
2. Open the Security tab.
3. Click the Security Group.
4. Verify the inbound rule.

Type	Port	Source
Custom TCP	8080	0.0.0.0/0
SSH	22	Your configured admin CIDR, if enabled

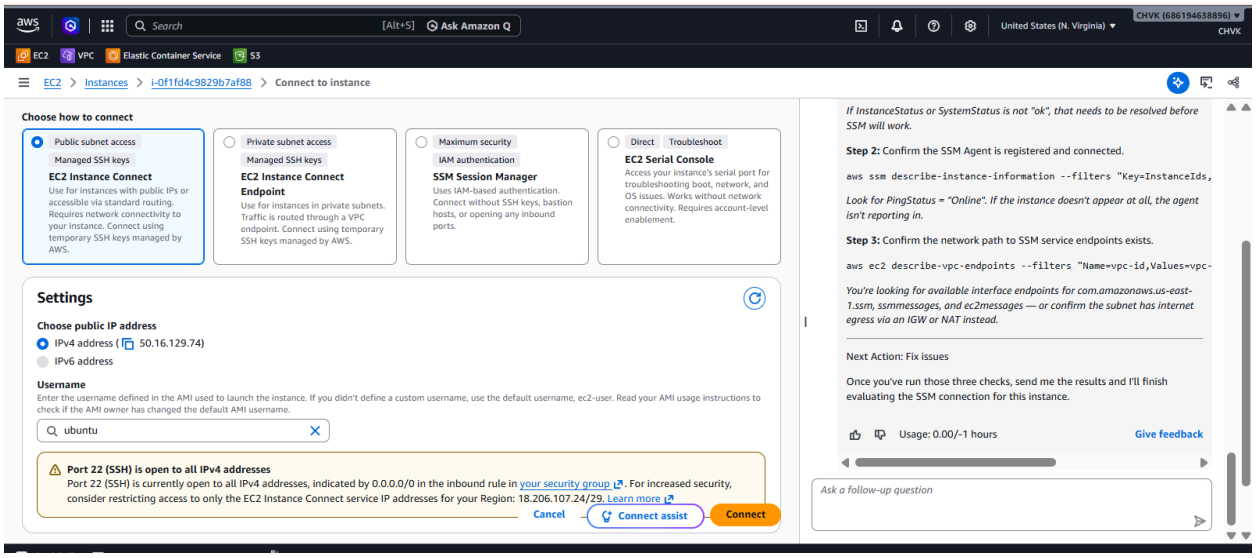
Port 8000 should not have a public inbound rule.

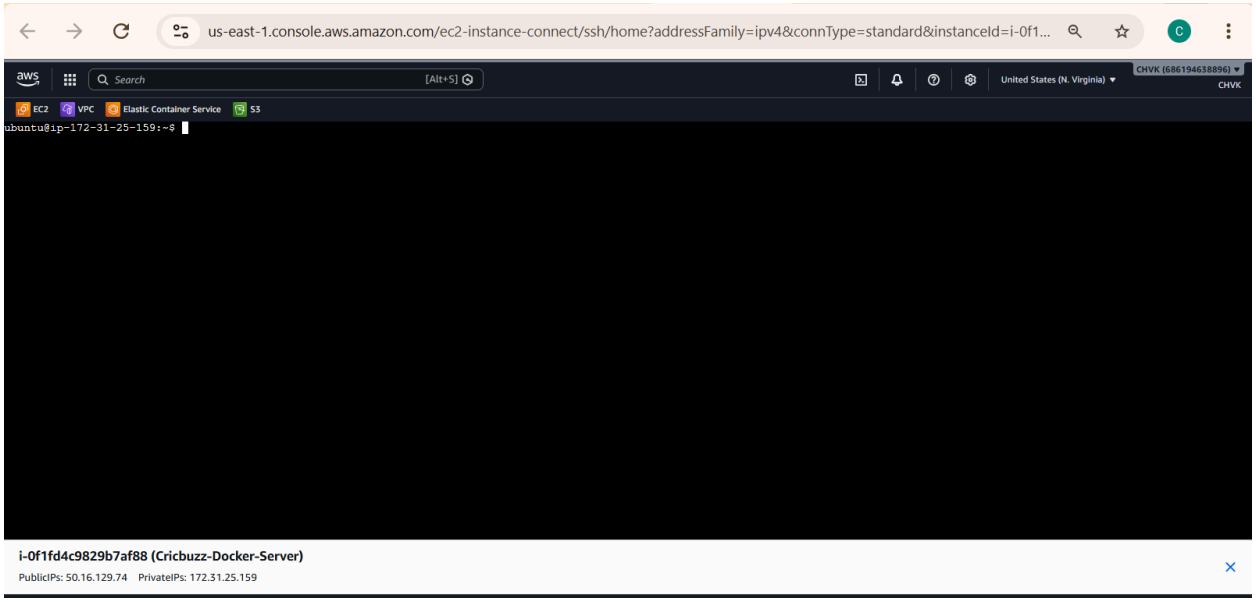


13. Connect to the EC2 instance

Open the instance `Cricbuzz-Docker-Server` on top there is an option connect click on that and select ec2 instance connect

It will open the terminal





Step 1: Check Docker

None

```
docker --version
```

Expected: Docker version information.

```
ubuntu@ip-172-31-25-159:~$ docker --version
Docker version 29.9.2, build a82e528
ubuntu@ip-172-31-25-159:~$
```

Step 2: Check Docker Compose

None

```
docker compose version
```

Expected: Docker Compose version information.

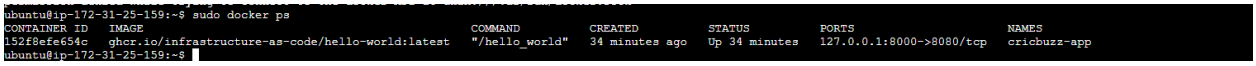
```
ubuntu@ip-172-31-25-159:~$ docker compose version
Docker Compose version v5.6.1
ubuntu@ip-172-31-25-159:~$
```

Step 3: Check running containers

```
None
sudo docker ps
```

Expected output should contain:

```
None
CONTAINER ID   IMAGE
xxxxxxxx       ghcr.io/infrastructure-as-code/hello-world
```

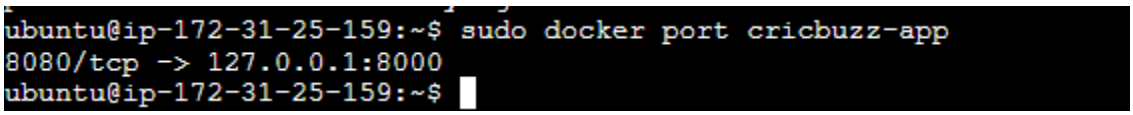


Check the published port:

```
None
sudo docker port cricbuzz-app
```

Expected:

```
None
8080/tcp -> 127.0.0.1:8000
```



Step 4: Check Nginx

```
None
sudo systemctl status nginx
```

Expected:

```
None
Active: active (running)
```

```
ubuntu@ip-172-31-25-159:~$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
     Loaded: loaded (/usr/lib/systemd/system/nginx.service; enabled; preset: enabled)
     Active: active (running) since Mon 2026-11-02 06:49:17 UTC; 35min ago
 Invocation: de2eefa08eb34374bff328f99086042c
       Docs: man:nginx(8)
    Process: 8186 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
    Process: 8188 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
   Main PID: 8189 (nginx)
      Tasks: 2 (limit: 657)
     Memory: 2.3M (peak: 3.1M)
        CPU: 21ms
     CGroup: /system.slice/nginx.service
             ├─8189 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             └─8190 "nginx: worker process"

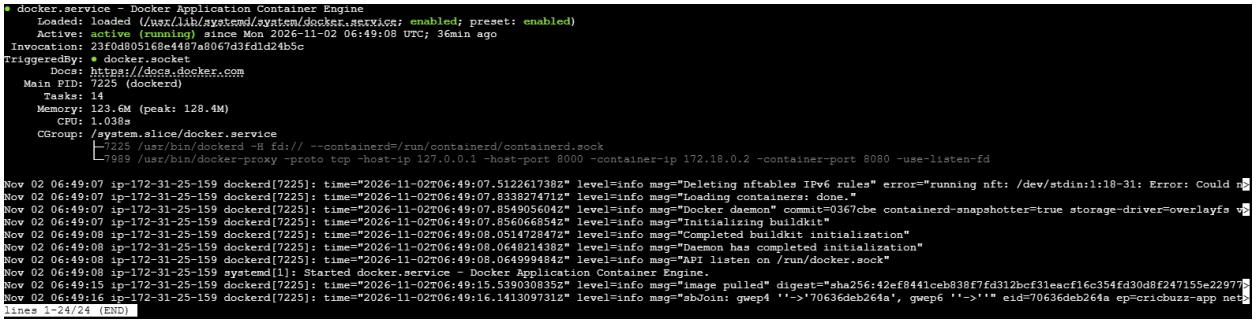
Nov 02 06:49:17 ip-172-31-25-159 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Nov 02 06:49:17 ip-172-31-25-159 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
ubuntu@ip-172-31-25-159:~$
```

Step 5: Check Docker

```
sudo systemctl status docker
```

Expected:

```
Active: active (running)
```



Step 6: Check Logrotate

```
logrotate --version
```

Expected: Logrotate version information.

```
ubuntu@ip-172-31-25-159:~$ logrotate --version
logrotate 3.22.0

    Default mail command:       /usr/bin/mail
    Default compress command:   /bin/gzip
    Default uncompress command: /bin/gunzip
    Default compress extension: .gz
    Default state file path:    /var/lib/logrotate/status
    ACL support:                yes
    SELinux support:            yes
ubuntu@ip-172-31-25-159:~$
```

14. Test the application

Now we will check whether Nginx forwards requests to the Docker application.

Test 1: Check Docker directly

Execute on EC2:

```
None
curl http://127.0.0.1:8000
```

Expected response:

```
None
Hello, World!
```

This confirms that the Docker application is running.

```
ubuntu@ip-172-31-25-159:~$ curl http://127.0.0.1:8000
Hello, World!ubuntu@ip-172-31-25-159:~$
```

Test 2: Check Nginx reverse proxy

Execute:

```
None
curl http://127.0.0.1:8080
```

Expected response:

```
None
Hello, World!
```

This confirms that Nginx is forwarding requests to Docker.

```
Hello, World!ubuntu@ip-172-31-25-159:~$ curl http://127.0.0.1:8080
Hello, World!ubuntu@ip-172-31-25-159:~$
```

Test 3: Check from your browser

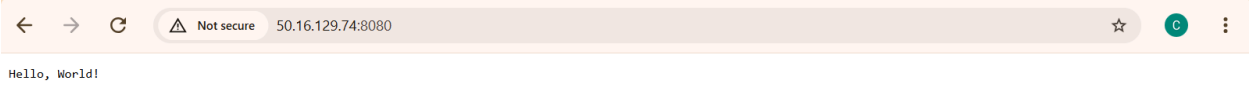
Open your browser on your local computer.

Enter:

```
None
http://YOUR-EC2-PUBLIC-IP:8080
```

Expected result: Hello, World!
[Hello world HTTP server example · YourBasic Go
yourbasic.org]

You should receive the same response from the Docker application.



Test 4: Test the health endpoint

```
None
curl -i http://127.0.0.1:8080/health
```

Expected: HTTP 200 response.

```
Hello, World!ubuntu@ip-172-curl -i http://127.0.0.1:8080/health1:8080/health
HTTP/1.1 200 OK
Server: nginx/1.28.3 (Ubuntu)
Date: Mon, 02 Nov 2026 07:29:29 GMT
Content-Type: text/plain; charset=utf-8
Content-Length: 0
Connection: keep-alive
Access-Control-Allow-Origin: *
X-Hostname: 152f8efe654c
```

Test 5: Test the Prometheus metrics endpoint

```
None
curl http://127.0.0.1:8080/metrics
```

Expected: Prometheus metrics information.

```
ubuntu@ip-172-31-25-159:~$ curl http://127.0.0.1:8080/metrics
# HELP gin_request_duration_seconds The HTTP request latencies in seconds.
# TYPE gin_request_duration_seconds summary
gin_request_duration_seconds_sum 0.000149364
gin_request_duration_seconds_count 6
# HELP gin_request_size_bytes The HTTP request sizes in bytes.
# TYPE gin_request_size_bytes summary
gin_request_size_bytes_sum 1719
gin_request_size_bytes_count 6
# HELP gin_requests_total How many HTTP requests processed, partitioned by status code and HTTP method.
# TYPE gin_requests_total counter
gin_requests_total{code="200",handler="main.healthFunc",host="127.0.0.1",method="GET",url="/health"} 1
gin_requests_total{code="200",handler="main.helloFunc",host="127.0.0.1",method="GET",url="/"} 1
gin_requests_total{code="200",handler="main.helloFunc",host="127.0.0.1:8000",method="GET",url="/"} 1
gin_requests_total{code="200",handler="main.helloFunc",host="50.16.129.74",method="GET",url="/"} 1
gin_requests_total{code="200",handler="main.helloFunc",host="50.16.129.74",method="GET",url="/?utm_source=chatgpt.com"} 1
gin_requests_total{code="404",handler="main.setupRouter.func1",host="50.16.129.74",method="GET",url="/favicon.ico"} 1
# HELP gin_response_size_bytes The HTTP response sizes in bytes.
# TYPE gin_response_size_bytes summary
gin_response_size_bytes_sum 51
gin_response_size_bytes_count 6
# HELP go_gc_duration_seconds A summary of the pause duration of garbage collection cycles.
# TYPE go_gc_duration_seconds summary
go_gc_duration_seconds{quantile="0"} 0
go_gc_duration_seconds{quantile="0.25"} 0
go_gc_duration_seconds{quantile="0.5"} 0
go_gc_duration_seconds{quantile="0.75"} 0
go_gc_duration_seconds{quantile="1"} 0
go_gc_duration_seconds_sum 0
```

15. Verify application logs

This is an important requirement in your assignment.

We configured Nginx to save access logs at:

```
None
/var/log/cb.log
```

Step 1: Generate some requests

Run:

```
None
curl http://127.0.0.1:8080
curl http://127.0.0.1:8080/health
curl http://127.0.0.1:8080/metrics
```

Step 2: Check the log file

```
None
sudo cat /var/log/cb.log
```

Expected log format:

```
None
127.0.0.1 - - [02/Nov/2026:10:30:00 +0000] "GET / HTTP/1.1" 200 ...
127.0.0.1 - - [02/Nov/2026:10:31:00 +0000] "GET /health HTTP/1.1"
200 ...
```

Your actual timestamps will differ.

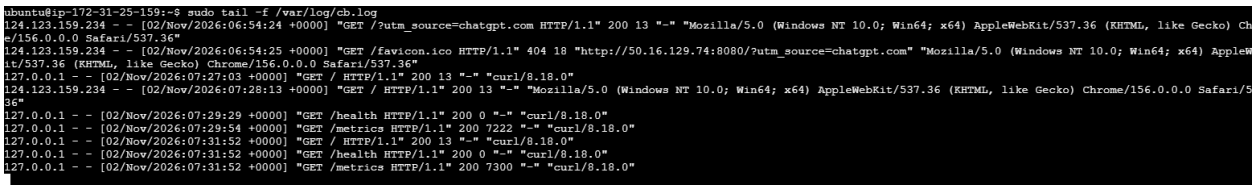


Step 3: Continuously monitor logs

```
None
sudo tail -f /var/log/cb.log
```

Now open the application in your browser.

You should see new requests appearing in the terminal.



Press `Ctrl + C` to stop monitoring.

Why do we save logs?

Logs help DevOps engineers:

- Identify application requests.
- Troubleshoot HTTP errors.
- Monitor user traffic.
- Investigate unexpected behavior.
- Understand application availability.

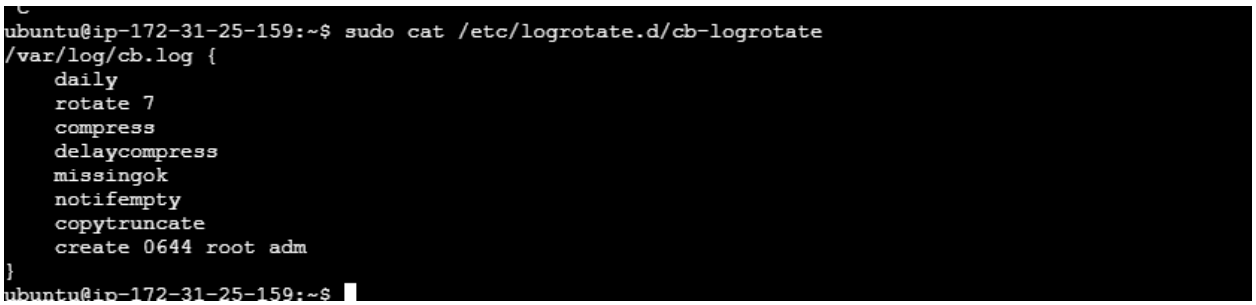
16. Verify Logrotate configuration

We have configured Logrotate at:

```
None
/etc/logrotate.d/cb-logrotate
```

Step 1: View configuration

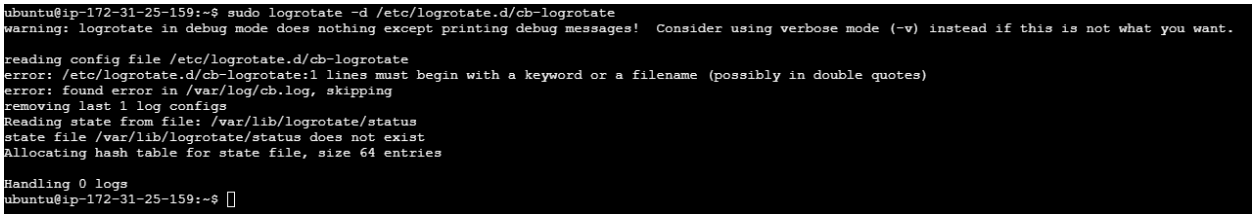
```
None
sudo cat /etc/logrotate.d/cb-logrotate
```



Step 2: Test configuration without changing logs

```
None
sudo logrotate -d /etc/logrotate.d/cb-logrotate
```

The `-d` option performs a debug/dry run.



Step 3: Force log rotation for testing

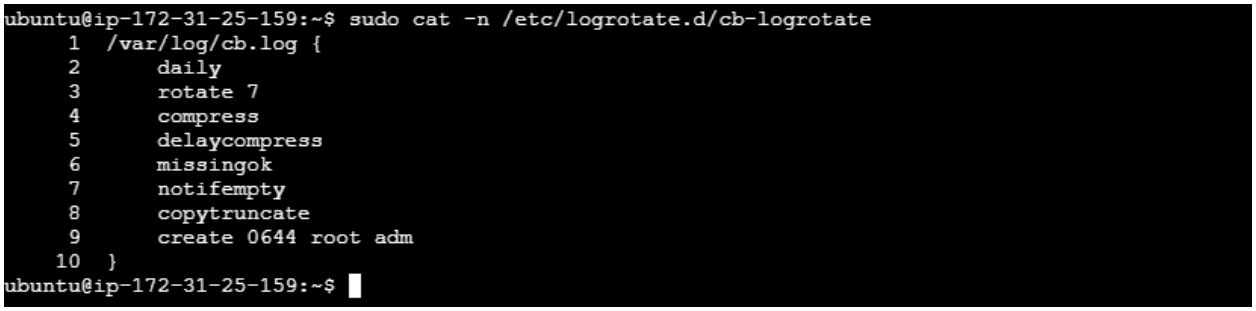
First generate some requests:

```
None
for i in {1..20}; do
  curl -s http://127.0.0.1:8080 > /dev/null
done
```

Now execute:

```
None
sudo logrotate -f /etc/logrotate.d/cb-logrotate
```

The `-f` option forces rotation for testing.



Step 4: Check rotated files

```
None
ls -lh /var/log/cb.log*
```

Expected structure:

```
None
cb.log
cb.log.1
```

```
ubuntu@ip-172-31-25-159:~$ ls -lh /var/log/cb.log*
-rw-r--r-- 1 root adm 2.9K Nov  2 07:35 /var/log/cb.log
ubuntu@ip-172-31-25-159:~$
```

After subsequent scheduled rotations, compressed files will also appear.

Because we use `delaycompress`, the most recently rotated file is not compressed immediately.

Note: Forced rotation is for testing. Normal rotation is scheduled daily. Logrotate retains up to seven rotated copies according to our configuration.

17. Troubleshooting – Common errors

Problem	Possible reason	Solution
EC2 not created	Incorrect AMI or region	Verify AMI availability.
Docker not installed	Installation script failed	Check cloud-init logs.
Container not running	Docker image pull failed	Check Docker logs and internet connectivity.
Port 8080 not accessible	Security Group issue	Verify inbound port 8080.
Nginx returns 502	Docker application unavailable	Check `docker ps` and port 8000.

Log file missing	Nginx configuration issue	Run `nginx -t` and check Nginx status.
Logs not rotating	Incorrect Logrotate configuration	Run Logrotate debug mode.

Important troubleshooting commands

Check the EC2 user-data execution log:

```
None
sudo tail -100 /var/log/cloud-init-output.log
```

Check Docker container logs:

```
None
docker logs cricbuzz-app
```

Check Nginx configuration:

```
None
sudo nginx -t
```

Check Nginx error logs:

```
None
sudo tail -50 /var/log/cb-error.log
```

Check whether port 8080 is listening:

```
None
sudo ss -tulnp | grep 8080
```

Check whether port 8000 is listening:

```
None
sudo ss -tulnp | grep 8000
```

18. Verify automatic installation completion

Terraform can finish creating the EC2 instance before the user-data script finishes installing Docker and Nginx.

Therefore, wait until cloud-init finishes.

Execute:

```
None
cloud-init status --wait
```

Expected:

```
None
status: done
```

Then verify the deployment completion message:

```
None
sudo cat /var/log/cricbuzz-deployment.log
```

Expected:

```
None
Cricbuzz deployment completed successfully
```

If this file does not exist, inspect:

```
None
sudo tail -100 /var/log/cloud-init-output.log
```

19. Complete project workflow

This is the complete workflow you should be able to explain in your DevOps interview or project demonstration.

Deployment workflow

1. Terraform initialization
   Downloads the AWS provider.
2. AWS infrastructure creation
   Creates Security Group and EC2 instance.
3. Ubuntu configuration
   EC2 User Data installs Docker, Docker Compose, Nginx and Logrotate.
4. Docker deployment
   Docker Compose downloads and starts the Hello World application.
5. Nginx configuration
   Nginx listens on port 8080 and forwards requests to port 8000.
6. Logging configuration
   Nginx stores access logs in `/var/log/cb.log`.
7. Log rotation
   Logrotate retains and compresses previous log files.
8. Application verification
   Browser returns the Docker application's response through port 8080.

20. Important: Clean up AWS resources

After completing your practice, if you do not need the EC2 instance anymore, open your VS Code terminal and run:

```
None
terraform destroy
```

Enter:

```
None
yes
```

```
# aws_security_group.cricbuzz_sg will be destroyed
- resource "aws_security_group" "cricbuzz_sg" {
    - arn                    = "arn:aws:ec2:us-east-1:686194638896:security-group/sg-0a689cae4677c726a" -> null
    - description            = "Allow HTTP application and restricted SSH" -> null
    - egress                 = [
        - {
            - cidr_blocks      = [
                - "0.0.0.0/0",
              ]
            - description      = "Allow outbound traffic"
            - from_port        = 0
            - ipv6_cidr_blocks = []
            - prefix_list_ids  = []
            - protocol         = "-1"
            - security_groups  = []
            - self             = false
            - to_port          = 0
          },
      ] -> null
    - id                     = "sg-0a689cae4677c726a" -> null
    - ingress                = [
        - {
            - cidr_blocks      = [
                - "0.0.0.0/0",
              ]
            - from_port        = 22
            - ipv6_cidr_blocks = []
            - prefix_list_ids  = []
            - protocol         = "tcp"
            - security_groups  = []
            - self             = false
            - to_port          = 22
              # (1 unchanged attribute hidden)
          },
        - {
            - cidr_blocks      = [
                - "0.0.0.0/0",
              ]
            - description      = "Nginx application port"
            - from_port        = 8080
            - ipv6_cidr_blocks = []
            - prefix_list_ids  = []
            - protocol         = "tcp"
            - security_groups  = []
            - self             = false
            - to_port          = 8080
          },
      ] -> null
```

```
Plan: 0 to add, 0 to change, 2 to destroy.

Changes to Outputs:
  - application_url = "http://50.16.129.74:8080" -> null
  - ec2_instance_id = "i-0f1fd4c9829b7af88" -> null
  - ec2_public_ip   = "50.16.129.74" -> null

Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

  Enter a value: yes

aws_instance.cricbuzz_server: Destroying... [id=i-0f1fd4c9829b7af88]
aws_instance.cricbuzz_server: Still destroying... [id=i-0f1fd4c9829b7af88, 00m10s elapsed]
aws_instance.cricbuzz_server: Still destroying... [id=i-0f1fd4c9829b7af88, 00m20s elapsed]
aws_instance.cricbuzz_server: Destruction complete after 21s
aws_security_group.cricbuzz_sg: Destroying... [id=sg-0a689cae4677c726a]
aws_security_group.cricbuzz_sg: Destruction complete after 2s

Destroy complete! Resources: 2 destroyed.
PS C:\terraform-cricbuzz-project>
```

This removes the EC2 instance and Security Group created by this Terraform configuration.

Remember: Destroying the instance also removes the application and its locally stored logs.

21. Interview explanation – How to explain this project

If an interviewer asks, "Explain your Terraform Docker deployment project", you can answer:

Sample interview answer

I created an AWS EC2 instance using Terraform with an Ubuntu AMI and t2.micro instance type.

I used EC2 User Data to automate the installation of Docker, Docker Compose, Nginx and Logrotate.

I deployed a Hello World application using Docker Compose from the provided Cricbuzz GitHub project.

The Docker application runs on container port 8080, which is mapped to localhost port 8000 on the EC2 instance.

I configured Nginx as a reverse proxy to listen on port 8080 and forward incoming requests to the Docker application on port 8000.

I also configured Nginx access logging to store request information in `/var/log/cb.log`.

Finally, I configured Logrotate to rotate and compress old log files and verified the complete application using the EC2 public IP.

22. Reference documentation

These are the official and project-specific references used for this implementation:

1. Cricbuzz Docker Hello World GitHub repository
2. Terraform documentation
3. AWS EC2 User Data documentation
4. Docker Compose documentation
5. Nginx reverse proxy documentation
6. Ubuntu Logrotate documentation

My recommendation: Complete this project in the exact order above. First create the seven files in VS Code, then execute `terraform init`, `terraform validate`, `terraform plan` and `terraform apply`. Verify the application and logs before moving to the next task.